Análise de Sentimentos

A ideia é simples: temos uma lista de palavras "positivas" e outra de "negativas". Para cada frase contamos quantas palavras de cada tipo aparecem e decidimos o sentimento pelo **saldo** (quem tiver mais, vence; empate = neutro).

1) O léxico: nossas listas de palavras

Tudo começa com duas listas simples: palavras que soam **positivas** e palavras que soam **negativas**.

In [1]:
```
palavras_positivas = ["bom", "boa", "ótimo", "excelente", "gostei", "adorei", "feliz"]
palavras_negativas = ["ruim", "péssimo", "horrível", "odiei", "triste", "chato"]
```

2) As frases que vamos analisar

Cinco frases de exemplo. Repare que a última não tem palavra positiva nem negativa.

In [2]:
```
frases = [
    "O produto é ótimo e eu adorei a entrega",
    "Que serviço péssimo, odiei o atendimento",
    "O filme foi bom mas o final foi ruim",
    "Estou muito feliz com a compra",
    "O relatório foi entregue",
]
```

3) Pré processamento

Antes do loop, vamos processar **só a primeira frase**. O truque é deixar tudo em minúsculo e quebrar o texto em uma lista de palavras.

In [3]:
```
frase = frases[0]

palavras = frase.lower().split()
palavras
```

Out[3]:
```
['o', 'produto', 'é', 'ótimo', 'e', 'eu', 'adorei', 'a', 'entrega']
```

4) Contando as palavras positivas e negativas

Percorremos palavra por palavra e somamos um ponto para cada lado.

In [4]:
```
positivos = 0
negativos = 0

for palavra in palavras:
    if palavra in palavras_positivas:
        positivos += 1
    elif palavra in palavras_negativas:
        negativos += 1

print("positivos:", positivos, "| negativos:", negativos)
```

```
positivos: 2 | negativos: 0
```

5) Decidindo o sentimento pelo saldo

Quem tiver mais pontos vence. Empate = neutro.

In [5]:
```
if positivos > negativos:
    sentimento = "Positivo"
elif negativos > positivos:
    sentimento = "Negativo"
else:
    sentimento = "Neutro"

print(sentimento)
```

```
Positivo
```

6) 5 frases

Agora que a lógica funciona para uma frase, é só colocá-la dentro de um `for` que passa por todas as frases.

In [6]:
```
for frase in frases:
    palavras = frase.lower().split()

    positivos = 0
    negativos = 0
    for palavra in palavras:
        if palavra in palavras_positivas:
            positivos += 1
        elif palavra in palavras_negativas:
            negativos += 1
```

```
    if positivos > negativos:
        sentimento = "Positivo"
    elif negativos > positivos:
        sentimento = "Negativo"
    else:
        sentimento = "Neutro"

    print(f"{sentimento:8} | +{positivos} / -{negativos} | {frase}")
```

```
Positivo | +2 / -0 | O produto é ótimo e eu adorei a entrega
Negativo | +0 / -1 | Que serviço péssimo, odiei o atendimento
Neutro   | +1 / -1 | O filme foi bom mas o final foi ruim
Positivo | +1 / -0 | Estou muito feliz com a compra
Neutro   | +0 / -0 | O relatório foi entregue
```

7) Criando uma função

In [7]:

```
def classificar_sentimento(frase):
    # Léxico simples de palavras em português
    palavras_positivas = [
        "bom", "boa", "ótimo", "excelente", "maravilhoso", "gostei", "gosto",
        "feliz", "amor", "adoro", "sensacional", "perfeito", "parabéns", "sucesso"
    ]
    palavras_negativas = [
        "ruim", "péssimo", "horrível", "odiei", "odeio", "triste", "pena",
        "chato", "difícil", "problema", "erro", "fracasso", "mal", "pior"
    ]

    # 1. Pré-processamento: minúsculas + remoção de pontuação
    frase_limpa = frase.lower()
    for pontuacao in [".", ",", "!", "?", ";", ":"]:
        frase_limpa = frase_limpa.replace(pontuacao, "")
    palavras = frase_limpa.split()

    # 2. Contagem de palavras
    positivos = 0
    negativos = 0
    for palavra in palavras:
        if palavra in palavras_positivas:
            positivos += 1
        elif palavra in palavras_negativas:
            negativos += 1

    # 3. Classificação pelo saldo
    if positivos > negativos:
```

```
        sentimento = "Positivo"
    elif negativos > positivos:
        sentimento = "Negativo"
    else:
        sentimento = "Neutro"

    # 4. Mostra a saída no mesmo formato do passo anterior
    print(f"{sentimento:8} | +{positivos} / -{negativos} | {frase}")
```

In [8]:
```
classificar_sentimento("O atendimento foi excelente, adorei a experiência!")
```

```
Positivo | +1 / -0 | O atendimento foi excelente, adorei a experiência!
```

In [9]:
```
classificar_sentimento("O atendimento foi excelente, adorei a experiência!")
```

```
Positivo | +1 / -0 | O atendimento foi excelente, adorei a experiência!
```

Alguns Desafios

In [10]:
```
classificar_sentimento("Não gostei da cor do produto.")
```

```
Positivo | +1 / -0 | Não gostei da cor do produto.
```

In [11]:
```
classificar_sentimento("Ah, que ótimo, quebrou no primeiro dia!")
```

```
Positivo | +1 / -0 | Ah, que ótimo, quebrou no primeiro dia!
```

In [12]:
```
classificar_sentimento("Seria bom se não travasse")
```

```
Positivo | +1 / -0 | Seria bom se não travasse
```

In [13]:
```
classificar_sentimento("Não tive problema nenhum")
```

```
Negativo | +0 / -1 | Não tive problema nenhum
```

In [14]:
```
classificar_sentimento("Mal posso esperar para usar")
```

```
Negativo | +0 / -1 | Mal posso esperar para usar
```

In [15]:
```
classificar_sentimento("Jogo difícil de largar, é muito viciante!!!")
```

```
Negativo | +0 / -1 | Jogo difícil de largar, é muito viciante!!!
```

Como podemos melhorar isso?

In [16]:
```
#!pip install transformers torch -q

from transformers import pipeline
```

```
classificador_t_pipeline = pipeline(
    "sentiment-analysis",
    model="lxyuan/distilbert-base-multilingual-cased-sentiments-student"
)
```

```
Loading weights: 100%                                                    104/104 [00:00<00:00, 6406.24it/s]
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
```

In [17]:
```
classificador_t_pipeline("Eu gostei muito do atendimento, foi excelente!")
```

Out[17]:
```
[{'label': 'positive', 'score': 0.9827386736869812}]
```

In [18]:
```
classificador_t_pipeline('Mal posso esperar para usar')
```

Out[18]:
```
[{'label': 'negative', 'score': 0.5004664659500122}]
```

In [19]:
```
classificador_t_pipeline("O atendimento foi excelente, adorei a experiência!")
```

Out[19]:
```
[{'label': 'positive', 'score': 0.9904999136924744}]
```

Como essa lib classifica os sentimentos?

Fluxo

```
Texto
↓
Tokens
↓
Embeddings
↓
Transformer
↓
Logits
↓
Softmax
↓
Classe + Confiança
```

1. Tokenização

O texto é dividido em **tokens** (palavras ou partes de palavras).

Exemplo:

```
"Gostei muito!"
↓
["Gostei", "muito", "!"]
```

---

2. Embeddings

Cada token é convertido em um **vetor numérico** que representa seu significado.

---

3. Transformer

O modelo analisa o **contexto** entre as palavras.

Exemplo:

```
"Não gostei."
```

Mesmo que **"gostei"** seja uma palavra positiva, o modelo percebe que **"Não"** muda o significado da frase para negativo.

---

4. Logits

O modelo calcula uma pontuação para cada classe.

Classe	Logit
Positivo	8.3
Neutro	1.4
Negativo	-2.8

Esses valores **não são probabilidades**.

---

5. Softmax

A função **Softmax** transforma os logits em probabilidades.

Classe	Probabilidade
Positivo	99.7%
Neutro	0.2%
Negativo	0.1%

A classe com maior probabilidade é escolhida.

Resultado

```
{
    "label": "positive",
    "score": 0.997
}
```

- **label:** sentimento previsto pelo modelo.
- **score:** confiança do modelo nessa previsão (99,7% neste exemplo).

In [ ]: